TECH REFERENCE

Temporal Ensemble Expert Technical Reference

Mamba SSM · Liquid Neural Network · PatchTST · Ensemble Gating

AIOps PLE for Financial Recommendation

v1.0 · 2026-04-01

> This document describes the mathematical structure, Ensemble Gating Mechanism, and Project Implementation Specification of the three models comprising the Temporal Ensemble Expert — Mamba (Selective State Space Model), Liquid Neural Network, and PatchTST (Patch Time Series Transformer).

Contents

1 Compound Structure of Financial Time Series . . . 3
1.1 Static Features vs. Temporal Features . . . 3
1.2 Three Components of Time Series . . . 3
1.3 Four Characteristics of Financial Transaction Data . . . 4
1.4 Window-Based vs. Event-Based Hybrid . . . 4
2 Mamba: Selective State Space Model . . . 4
2.1 Theoretical Background . . . 4
2.2 Continuous State Space Model . . . 4
2.3 ZOH Discretization . . . 5
2.4 Discrete Recurrence . . . 5
2.5 S6 Selective Mechanism . . . 5
2.5.1 Selective Mechanism in Financial Domain . . . 5
2.6 Selective Sequential Scan . . . 5
2.7 Gated MLP + Causal Conv1d Architecture . . . 6
2.8 A Matrix Initialization and Stability . . . 6
2.9 Project Implementation Specification . . . 6
2.9.1 Transaction Mamba . . . 6
2.9.2 Session Mamba . . . 7
2.10 Complexity Analysis . . . 7
3 Liquid Neural Network (LNN) . . . 7
3.1 Theoretical Background . . . 7
3.2 Core ODE . . . 7
3.2.1 Physical Interpretation of Each Term . . . 8
3.3 Euler Discretization . . . 8
3.4 Financial Domain Interpretation of the Adaptive Time Constant . . . 8
3.5 SingleStep Mode Design . . . 8
3.5.1 Design Rationale . . . 8
3.6 Project Implementation Specification . . . 9
4 PatchTST: Patch Time Series Transformer . . . 9
4.1 Theoretical Background . . . 9
4.2 Patch Embedding . . . 9
4.2.1 Financial Domain Interpretation of Patch Size . . . 9
4.3 Multi-Head Self-Attention . . . 10
4.4 Positional Encoding . . . 10
4.5 Project Implementation Specification . . . 10
4.6 Complexity Comparison . . . 10
5 Ensemble Gating . . . 11
5.1 Gate Architecture . . . 11
5.2 Ensemble Diversity Through Input Separation . . . 11
5.3 Gate Entropy Monitoring . . . 11
5.4 Final Output Flow Summary . . . 12
6 I/O Specification and Implementation Notes . . . 12

6.1 Full I/O Specification 12
6.1.1 Inputs 12
6.1.2 Intermediate Representations 12
6.1.3 Outputs 13
6.2 Summary: Role Comparison of the Three Models 13
6.3 Generational Progression of Time Series Analysis and This Project 13
6.4 Implementation Considerations 13
6.4.1 Data Leakage Prevention 13
6.4.2 Numerical Stability 14
6.4.3 Performance Optimization 14
6.4.4 References 14

Compound Structure of Financial Time Series

Static Features vs. Temporal Features

In traditional recommendation systems, user representations are *static* vectors. They consist of time-independent properties such as age, occupation, and preferred categories. However, real user behavior changes continuously along the time axis, and reducing it to a single snapshot causes critical behavioral signals such as *periodicity*, *trends*, and *seasonality* to disappear.

Perspective	Static Features	Temporal Features
Representation	Fixed vector $\boldsymbol{x} \in \mathbb{R}^d$	Sequence $\boldsymbol{X} \in \mathbb{R}^{T \times d}$
Information Loss	Averaging over time axis → pattern loss	Order, interval, and trend preserved
Model Requirements	MLP, Embedding tables	SSM, ODE, Transformer and other sequence models

The Temporal Expert retains transaction and session data as sequences, learning patterns embedded in the temporal dimension and compressing them into a 64D representation.

Three Components of Time Series

Every time series $y(t)$ fundamentally decomposes into the sum of three components:

$$y(t) = T(t) + S(t) + R(t) \tag{1}$$

$T(t)$: Trend — long-term directional movement
$S(t)$: Seasonality — repetitive periodic patterns (daily, monthly, yearly)
$R(t)$: Residual — irregular variation not explained by trend and seasonality

The three models of the Temporal Expert correspond to this decomposition:

- **Mamba**: Trend (long-range dependencies, capturing directional movement through selective memory)
- **PatchTST**: Seasonality (capturing global periodicity through inter-patch attention)

- **LNN**: Residual (handling irregular variation with adaptive time constants)

Four Characteristics of Financial Transaction Data

Financial time series have unique characteristics that distinguish them from general time series:

Characteristic	Description	Processing Model
Snapshot discontinuity	Discrete observations such as month-end balances and quarterly reviews	Mamba selective memory
Daily transaction bursts	Dozens of payments and transfers per day	PatchTST patch aggregation
Irregular intervals	Business days vs. holidays, active vs. dormant periods	LNN adaptive τ
High-frequency events	Real-time card authorization streams	Mamba $O(L)$ linear processing

Window-Based vs. Event-Based Hybrid

In this project, transaction data is aggregated daily to form *window-based sequences* (180 timesteps). These are fed into Mamba and PatchTST. Simultaneously, *actual time interval* information between transactions is passed to LNN, performing event-based temporal awareness correction.

Approach	Window-Based (Fixed Interval)	Event-Based (Irregular Interval)
Time Model	$\Delta t = \text{const}$	$\Delta t_i \neq \Delta t_j$
Data Shape	$\boldsymbol{X} \in \mathbb{R}^{T \times d}$	$\{(t_i, \boldsymbol{x}_i)\}$ event stream
Suitable Model	Mamba, PatchTST	LNN (ODE-based)

Mamba: Selective State Space Model

Theoretical Background

> **Reference Paper**
>
> Gu & Dao, *"Mamba: Linear-Time Sequence Modeling with Selective State Spaces"* (NeurIPS 2023).
> Lineage: HiPPO (Gu et al., NeurIPS 2020) → S4 (ICLR 2022) → Mamba (2023).

An architecture that extends SSM (State Space Model) with an input-dependent selective mechanism (S6), combining the linear time complexity of RNNs with the content-aware capabilities of Transformers.

Continuous State Space Model

Basic form of a linear time-invariant (LTI) system:

$$\frac{d\boldsymbol{x}}{dt} = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{B}u, \quad y = \boldsymbol{C}\boldsymbol{x} + \boldsymbol{D}u \tag{2}$$

$\boldsymbol{x} \in \mathbb{R}^N$: hidden state, $u \in \mathbb{R}$: input signal, $y \in \mathbb{R}$: output signal
$\boldsymbol{A} \in \mathbb{R}^{N\times N}$: state transition matrix, $\boldsymbol{B} \in \mathbb{R}^{N\times 1}$: input matrix, $\boldsymbol{C} \in \mathbb{R}^{1\times N}$: output matrix

ZOH Discretization

Converting the continuous system to discrete time steps Δ:

$$\bar{\boldsymbol{A}} = \exp(\Delta \cdot \boldsymbol{A}), \quad \bar{\boldsymbol{B}} \approx \Delta \cdot \boldsymbol{B} \tag{3}$$

$\bar{\boldsymbol{A}}$ represents "the state transition that advances exactly Δ time units in continuous time." The first-order approximation $\exp(\Delta\boldsymbol{A}) \approx \boldsymbol{I} + \Delta\boldsymbol{A}$ is the Euler method, while the exact matrix exponential accounts for infinitely many orders. When $\boldsymbol{A}$ is diagonal, $\exp(\boldsymbol{A}t) = \mathrm{diag}(e^{a_1 t}, ..., e^{a_N t})$, which is why Mamba restricts $\boldsymbol{A}$ to be diagonal.

Discrete Recurrence

$$\boldsymbol{h}_t = \bar{\boldsymbol{A}} \cdot \boldsymbol{h}_{t-1} + \bar{\boldsymbol{B}} \cdot \boldsymbol{x}_t, \quad \boldsymbol{y}_t = \boldsymbol{C}_t \cdot \boldsymbol{h}_t \tag{4}$$

Intuitively: "today's state = yesterday's memory × decay rate + today's new information."

S6 Selective Mechanism

In conventional LTI SSMs, $\boldsymbol{A}, \boldsymbol{B}, \boldsymbol{C}$ are constants independent of input. Mamba's S6 generates $\Delta, \boldsymbol{B}, \boldsymbol{C}$ *input-dependently* to achieve content-aware processing.

$$\Delta = \mathrm{softplus}(\boldsymbol{W}_\Delta \cdot \boldsymbol{x} + \boldsymbol{b}_\Delta) \tag{5}$$

$$\boldsymbol{B} = \boldsymbol{W}_B \cdot \boldsymbol{x}, \quad \boldsymbol{C} = \boldsymbol{W}_C \cdot \boldsymbol{x} \tag{6}$$

$\boldsymbol{W}_\Delta \in \mathbb{R}^{D\times r}$: Δ projection ($r = \lceil D/16 \rceil$, dt_rank)
softplus guarantees: $\Delta > 0$ (time steps are positive)

When Δ is large for a given input, the information at that timestep is strongly memorized (encoded); when Δ is small, the previous state is retained and the current input is ignored (forgotten).

Selective Mechanism in Financial Domain

- Large transaction → large Δ → strongly recorded in hidden state (selective memory)
- Small daily transaction → small Δ → previous state maintained, processed as background (selective forgetting)
- This mechanism naturally models the *heterogeneous importance* of financial events

Selective Sequential Scan

Since $\Delta_t, \boldsymbol{B}_t, \boldsymbol{C}_t$ differ at each timestep:

$$\boldsymbol{h}_t = \underbrace{\exp(\Delta_t \cdot \boldsymbol{A})}_{\bar{\boldsymbol{A}}_t} \cdot \boldsymbol{h}_{t-1} + \underbrace{\Delta_t \cdot \boldsymbol{B}_t}_{\bar{\boldsymbol{B}}_t} \cdot \boldsymbol{x}_t, \quad \boldsymbol{y}_t = \boldsymbol{C}_t^\top \boldsymbol{h}_t \tag{7}$$

The entire sequence is processed in $O(L)$ linear time complexity.

Gated MLP + Causal Conv1d Architecture

MambaBlock wraps the SSM with a Gated MLP and 1D causal convolution:

1. **Input Projection**: `d_input` → `d_model`
2. **LayerNorm → in_proj**: `d_model` → `2 * d_inner` (split into two paths)
3. **SSM path**: Causal Conv1d (kernel=4) → SiLU → SelectiveSSM (S6)
4. **Gate path**: SiLU activation
5. **Element-wise multiply** (SSM output ∘ Gate output)
6. **out_proj**: `d_inner` → `d_model` + Residual connection

The causal Conv1d (kernel=4) references only $t-3, t-2, t-1, t$ at timestep t, mixing local context without leaking future information.

A Matrix Initialization and Stability

$\boldsymbol{A}$ is stored in log space and restored as $-\exp(\mathrm{A}_{\log})$ during the forward pass. This guarantees that all eigenvalues are *always negative*, ensuring system stability.

The initial value is a HiPPO-style diagonal $[1, 2, ..., N]$. States with larger indices correspond to higher-frequency polynomial components, so diagonal elements become $-1, -2, ..., -N$, causing higher-frequency components to decay faster.

Project Implementation Specification

Transaction Mamba

Parameter	Value	Description
`d_model`	128	hidden dimension
`d_input`	16	card (8D) + deposit (8D) transaction features
`d_inner`	256	128×2 (expand=2)
`d_state`	16	SSM state vector dimension
`d_conv`	4	1D causal convolution kernel size
`dt_rank`	8	$\lceil 128/16 \rceil = 8$
`seq_len`	180	180-day transaction sequence
Output	`[B, 128]`	last timestep `[:, -1, :]`

Session Mamba

Parameter	Value	Description
`d_model`	64	half dimension compared to transaction
`d_input`	8	session feature dimension
`d_inner`	128	64×2 (expand=2)
`seq_len`	90	90-day session sequence
Output	`[B, 64]`	last timestep

Total Mamba output dimension: $128 + 64 = 192$D (txn + session concat).

Complexity Analysis

Operation	Complexity	Notes
Sequential scan	$O(L \cdot D \cdot N)$	L: sequence length, D: model dimension, N: state dimension
vs. Transformer	$O(L)$ vs $O(L^2)$	linear in sequence length
Memory	$O(D \cdot N)$	only hidden state retained (independent of sequence length)

Liquid Neural Network (LNN)

Theoretical Background

> **Reference Paper**
>
> Hasani et al., *"Liquid Time-constant Networks"* (AAAI 2021).
> Lineage: Neural ODE (Chen et al., NeurIPS 2018) + biological neuron decay model + Liquid State Machine (Maass et al., 2002).

A continuous-time neural network based on Neural ODE, where the time constant τ adapts dynamically according to the input. It can naturally handle irregular time intervals (an intrinsic characteristic of financial transactions).

Core ODE

$$\frac{d\boldsymbol{h}}{dt} = \frac{-\boldsymbol{h} + f(\boldsymbol{x}, \boldsymbol{h})}{\tau(\boldsymbol{x}, \boldsymbol{h})} \tag{8}$$

> $\boldsymbol{h} \in \mathbb{R}^H$: hidden state, $\boldsymbol{x}$: external input
> $f(\cdot,\cdot)$: state update function (target state), $\tau(\cdot,\cdot) > 0$: adaptive time constant

Physical Interpretation of Each Term

Term	Meaning
$-\boldsymbol{h}$	Decay (leak): hidden state converges to 0 without input. Naturally forgets old information.
$f(\boldsymbol{x}, \boldsymbol{h})$	Driving force: target state based on new input and current state. $f = \tanh\big(\boldsymbol{W}_f[\boldsymbol{x}; \boldsymbol{h}] + \boldsymbol{b}_f\big)$
$\tau(\boldsymbol{x}, \boldsymbol{h})$	Time constant: larger values mean slower change (inertia, state preservation), smaller values mean faster response. $\tau = \text{Softplus}(\text{MLP}([\boldsymbol{x}; \boldsymbol{h}])) + 0.1$

Euler Discretization

Converting the continuous ODE to discrete timesteps:

$$\boldsymbol{h}_{t+1} = \boldsymbol{h}_t + \Delta t \cdot \frac{-\boldsymbol{h}_t + f(\boldsymbol{x}_t, \boldsymbol{h}_t)}{\tau(\boldsymbol{x}_t, \boldsymbol{h}_t)} \tag{9}$$

Δt is the *actual time interval* (in days), so the ODE naturally handles irregular transaction intervals.

Financial Domain Interpretation of the Adaptive Time Constant

Transaction intervals in finance are extremely irregular:

Situation	Δt	Role of τ
Multiple transactions on the same day	~ 0.01 days	Small τ: fast response, immediately reflecting each transaction
Weekend gap	2 days	Medium τ: state maintained gradually
Long-term dormancy	> 30 days	Large τ: state preserved, slow decay

RNNs/LSTMs with fixed τ treat all intervals equally. The adaptive τ responds quickly during active transaction periods and preserves state during dormant periods, automatically adapting to *customer behavioral rhythms*.

SingleStep Mode Design

In this project, LNN operates in **SingleStep mode**. Rather than processing the entire sequence through ODE, it applies only a *single ODE step* to Mamba's final hidden state.

Design Rationale

- Mamba already captures the full sequence patterns in $O(L)$
- LNN serves only as a *temporal scale correction* role (avoiding redundant sequence processing)
- Computational cost: $O(1)$ single step vs. $O(L)$ full sequence ODE

Mamba → LNN Serial Structure

After Mamba learns sequence patterns, LNN applies temporal-aware correction to the final state in a serial structure. This corresponds to the decomposition of "trend extraction → residual correction."

Project Implementation Specification

Parameter	LNN txn	LNN session	Description
`input_dim`	128	64	Mamba output dimension
`hidden_dim`	64	32	LNN hidden dimension
Output	`[B, 64]`	`[B, 32]`	SingleStep output

Total LNN output dimension: $64 + 32 = 96$D (txn + session concat).

PatchTST: Patch Time Series Transformer

Theoretical Background

Reference Paper

Nie et al., *"A Time Series is Worth 64 Words: Long-term Forecasting with Transformers"* (ICLR 2023). Applies the patch concept from ViT (Dosovitskiy et al., ICLR 2021) to time series.

The time series is divided into fixed-size patches, and each patch is treated as a token with Self-Attention applied. This reduces the attention cost from $O(L^2)$ to $O((L/P)^2)$ while effectively capturing multi-scale periodic patterns.

Patch Embedding

$$\boldsymbol{p}_i = \boldsymbol{W}_{\text{proj}} \cdot \text{flatten}\Big(\boldsymbol{x}_{[((i-1)P+1):(iP)]}\Big) + \boldsymbol{b}_{\text{proj}} \tag{10}$$

With patch size $P = 16$, the 180-timestep transaction sequence is converted to $\lfloor 180/16 \rfloor = 11$ patches. Attention cost: $O(180^2) = 32400 \rightarrow O(11^2) = 121$.

Financial Domain Interpretation of Patch Size

A patch size of 16 corresponds to approximately 2 weeks, naturally aligning with the fundamental unit of payroll cycles (bi-weekly/monthly). Local patterns within each patch (daily spending) are aggregated, and inter-patch attention captures global periodicity (monthly salary, quarterly bonuses).

Multi-Head Self-Attention

$$\text{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \text{softmax}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^\top}{\sqrt{d_k}}\right)\boldsymbol{V} \tag{11}$$

The (i, j) element of $\boldsymbol{Q}\boldsymbol{K}^\top$ is the inner product of two patch vectors, proportional to *cosine similarity*. Softmax converts this to a probability distribution, computing a weighted sum of information from highly relevant patches. Division by $\sqrt{d_k}$ normalizes the inner product variance to maintain healthy softmax gradients.

Positional Encoding

Sinusoidal positional encoding is used. For 6–12 patches, it provides sufficient performance compared to learnable positional encoding and generalizes better to varying sequence lengths.

Project Implementation Specification

Parameter	PatchTST txn	PatchTST session	Description
`d_model`	64	32	Transformer hidden dimension
`nhead`	4	2	number of multi-head attention heads
`num_layers`	2	2	number of Transformer encoder layers
`patch_size`	16	16	patch size (approximately 2 weeks)
Patch count	11	5	$\lfloor L/P \rfloor$
Output	`[B, 64]`	`[B, 32]`	AdaptiveAvgPool1d

Total PatchTST output dimension: $64 + 32 = 96$D (txn + session concat).

Complexity Comparison

Model	Complexity	At L=180
Vanilla Transformer	$O(L^2)$	$32,400$
PatchTST ($P = 16$)	$O((L/P)^2)$	121
Mamba (SSM)	$O(L)$	180

Ensemble Gating

Gate Architecture

After concatenating the outputs of three models, a learnable 2-layer gate network dynamically determines each model's contribution:

$$\boldsymbol{g} = \text{Softmax}(\boldsymbol{W}_2 \cdot \text{ReLU}(\boldsymbol{W}_1 \cdot \boldsymbol{z}_{\text{cat}} + \boldsymbol{b}_1) + \boldsymbol{b}_2) \in \mathbb{R}^3 \tag{12}$$

$$\boldsymbol{y} = \sum_{i=1}^{3} g_i \cdot \text{Proj}_i(\boldsymbol{z}_i) \in \mathbb{R}^{64} \tag{13}$$

$\boldsymbol{z}_{\text{cat}}$: concatenation of three model outputs (192 + 96 + 96 = 384D)
Proj_i: projects each model output into a common 64D space
g_i: weight of model i ($\sum g_i = 1$)

Ensemble Diversity Through Input Separation

Model	Input Path	Basis for Diversity
Mamba	direct input of original sequence	sequential state space perspective
LNN	Mamba final state (serial)	temporal scale correction perspective
PatchTST	independent input of original sequence	global attention perspective

The Mamba → LNN serial + PatchTST independent structure ensures ensemble diversity through input separation. Using identical inputs reduces the differentiation effect of gating, making independent path design essential.

Gate Entropy Monitoring

The health of the gate distribution is monitored using Shannon entropy:

$$H(\boldsymbol{g}) = -\sum_{i=1}^{3} g_i \log_2(g_i) \tag{14}$$

State	Entropy	Interpretation
Uniform distribution	$\log_2(3) \approx 1.585$ bits	Three models contribute equally (maximum entropy)
Healthy skew	$0.5 \sim 1.2$ bits	Adaptive weights according to input (normal operation)
Gate Collapse	< 0.3 bits	One model dominates, others stop learning (warning)

Gate Collapse

When $H < 0.3$ bits, one model monopolizes almost all the weight. The remaining two models have their gradients blocked and stop learning, rendering the ensemble meaningless. Log entropy periodically during training, and apply gate temperature scaling or entropy bonus regularization when collapse is detected.

Final Output Flow Summary

$$\begin{aligned}
\text{txn_seq } [B, 180, 16] &\rightarrow \text{Mamba}_{\text{txn}} \rightarrow [B, 128] \\
\text{sess_seq } [B, 90, 8] &\rightarrow \text{Mamba}_{\text{sess}} \rightarrow [B, 64] \\
\text{Mamba}_{\text{txn}} &\rightarrow \text{LNN}_{\text{txn}} \rightarrow [B, 64] \\
\text{Mamba}_{\text{sess}} &\rightarrow \text{LNN}_{\text{sess}} \rightarrow [B, 32] \\
\text{txn_seq} &\rightarrow \text{PatchTST}_{\text{txn}} \rightarrow [B, 64] \\
\text{sess_seq} &\rightarrow \text{PatchTST}_{\text{sess}} \rightarrow [B, 32]
\end{aligned} \tag{15}$$

Concat: $[192 + 96 + 96 = 384\text{D}] \rightarrow$ Gate $\rightarrow$ Weighted sum $\rightarrow$ **64D** final output.

I/O Specification and Implementation Notes

Full I/O Specification

Inputs

Input	Shape	Description
`txn_seq`	`[B, 180, 16]`	180-day transaction sequence, card 8D + deposit 8D
`session_seq`	`[B, 90, 8]`	90-day session sequence, 8D session features

Intermediate Representations

Model	txn output	session output	Total
Mamba (SSM)	128D	64D	192D
LNN (ODE)	64D	32D	96D
PatchTST (Attn)	64D	32D	96D
Concat			**384D**

Outputs

Output	Shape	Description
Temporal Expert output	`[B, 64]`	final representation after gated weighted sum
Gate weights	`[B, 3]`	Mamba, LNN, PatchTST contributions (for monitoring)

The 64D output vector is passed to PLE's CGC Gate Attention, where it is dynamically combined per task with the outputs of the other 6 experts (PersLay, DeepFM, LightGCN, Unified H-GCN, Causal, OT).

Summary: Role Comparison of the Three Models

Model	Temporal Pattern Captured	Mechanism	Complexity
Mamba	Long-range sequential dependencies (trend)	Selective State Space (S6)	$O(L)$ linear
LNN	Irregular time intervals (residual)	Adaptive time constant ODE	$O(1)$ single step
PatchTST	Global periodicity (seasonality)	Patch-level Self-Attention	$O((L/P)^2)$

Generational Progression of Time Series Analysis and This Project

Generation	Approach	Limitation	Overcoming Model
1st	ARIMA, Exponential Smoothing	linearity assumption, manual differencing	LSTM, GRU
2nd	LSTM, GRU (gated RNNs)	$O(L)$ sequential bottleneck, vanishing gradient	Transformer
3rd	Transformer (Self-Attention)	$O(L^2)$ complexity, weak ordering information	SSM, PatchTST
4th	SSM + ODE + Patch Transformer	model complexity, Gate Collapse risk	this project

Implementation Considerations

Data Leakage Prevention

- `gap_days` must be set so the last timestep of sequence data does not overlap with the label derivation period (minimum 7 days)
- Mamba's causal Conv1d does not reference future information (left-padding)
- PatchTST attention references the entire sequence without a causal mask, but since the input sequence is already truncated to before the label period, there is no leakage risk

Numerical Stability

- Mamba $\boldsymbol{A}$ matrix: negativity guaranteed by $-\exp(\mathrm{A}_{\log})$ (prevents divergence)
- Mamba Δ: positivity guaranteed by softplus ($\Delta > 0$)
- LNN τ: lower bound of $\mathrm{Softplus}(\cdot) + 0.1$ (prevents zero division)
- Warning logged when gate entropy < 0.3 bits

Performance Optimization

- Mamba sequential scan: prototype uses Python loop; production recommends `mamba-ssm` CUDA kernel
- AMP (Mixed Precision) enabled: approximately 2x speed improvement on g4dn T4 GPU
- Batch size: adjusted based on VRAM and data scale (4096 recommended for large-scale data)

References

Component	Paper	Venue
Mamba SSM	Gu & Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces"	NeurIPS 2023
LNN	Hasani et al., "Liquid Time-constant Networks"	AAAI 2021
PatchTST	Nie et al., "A Time Series is Worth 64 Words"	ICLR 2023
Neural ODE	Chen et al., "Neural Ordinary Differential Equations"	NeurIPS 2018
S4 (SSM)	Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces"	ICLR 2022
HiPPO	Gu et al., "HiPPO: Recurrent Memory with Optimal Polynomial Projections"	NeurIPS 2020
ViT	Dosovitskiy et al., "An Image is Worth 16x16 Words"	ICLR 2021
CfC	Hasani et al., "Closed-form Continuous-depth Models"	Nature MI 2022